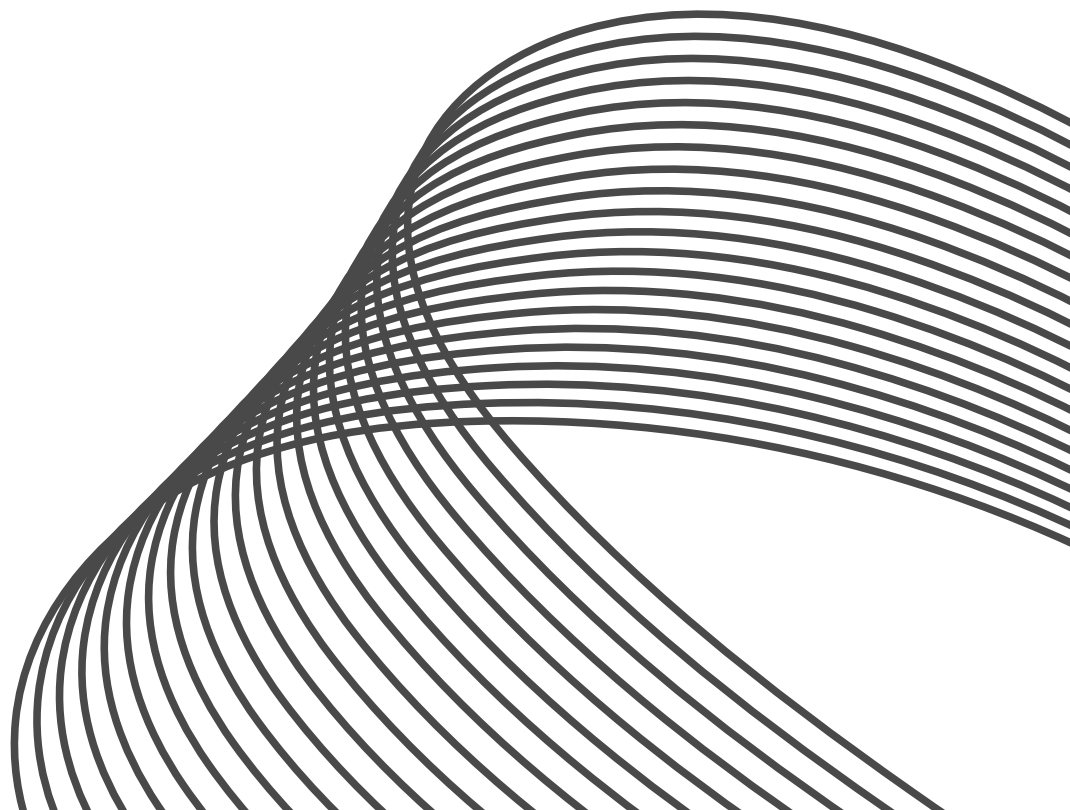


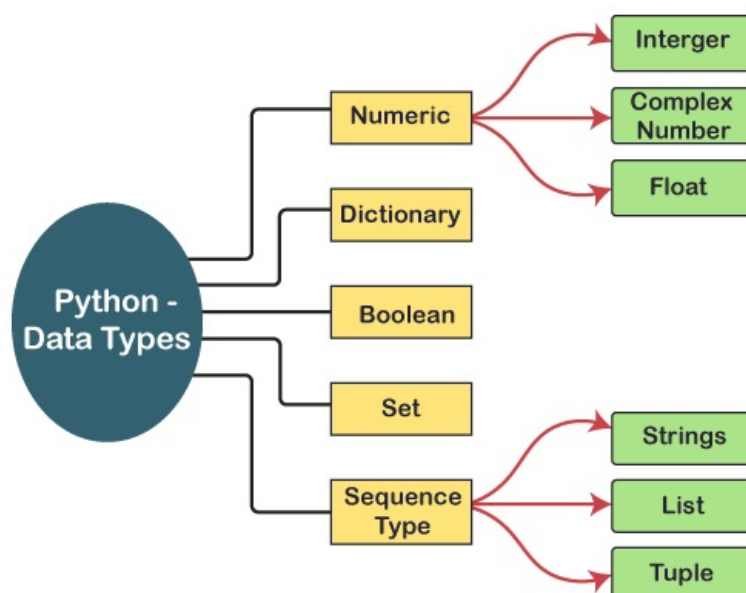
Python Basics



Welcome to the Python Basics



Different Types of Data Types in Python



There are eight kinds of types supported by PyTables:

- **bool**: Boolean (true/false) types. Supported precisions: 8 (default) bits.
- **int**: Signed integer types. Supported precisions: 8, 16, 32 (default) and 64 bits.
- **uint**: Unsigned integer types. Supported precisions: 8, 16, 32 (default) and 64 bits.
- **float**: Floating point types. Supported precisions: 16, 32, 64 (default) bits and extended precision floating point (see note on floating point types).
- **complex**: Complex number types. Supported precisions: 64 (32+32), 128 (64+64, default) bits and extended precision complex (see note on floating point types).
- **string**: Raw string types. Supported precisions: 8-bit positive multiples.
- **time**: Data/time types. Supported precisions: 32 and 64 (default) bits.

- **enum**: Enumerated types. Precision depends on base type

bool -> True, False / 1,0

int -> 3,5,-1,-9

float -> 3.45

str -> 'data scientist'

```
In [ ]: # bool, int, float, str
a = 'Data Science'
print(a)
type(a)
```

Data Science

Out[]: str

```
In [ ]: type(a)
```

Out[]: str

```
In [ ]: z = True
type(z)
```

Out[]: bool

```
In [ ]: #AutotYPEcasting
3 + 4.5
```

Out[]: 7.5

```
In [ ]: True + 2
```

Out[]: 3

```
In [ ]: int(5.1) #converting float to int
```

Out[]: 5

```
In [ ]: # bool -> int -> float -> string
bool(-18)
```

Out[]: True

```
In [ ]: bool(18) #anything which is not zero is "TRUE" in boolean
```

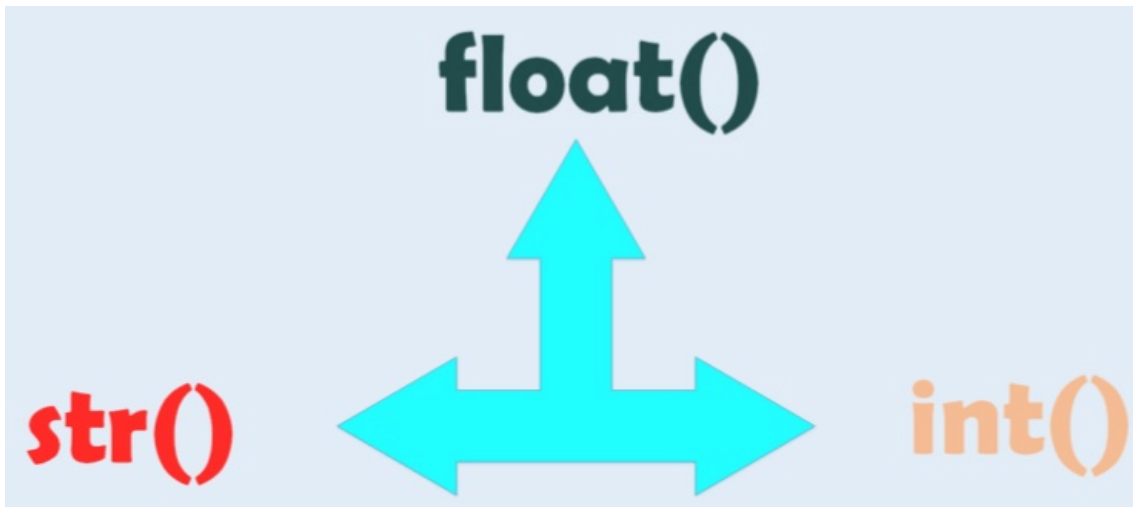
Out[]: True

```
In [ ]: name = 'vivek'
```

```
type(name)
```

```
Out[ ]: str
```

Type Casting:



The conversion of one data type into the other data type is known as type casting in python or type conversion in python.

```
In [ ]: 3 + 6.5
```

```
Out[ ]: 9.5
```

```
In [ ]: # bool -> int -> float -> str  
True + 6 + 7.5  
# 1+6+7.5
```

```
Out[ ]: 14.5
```

```
In [ ]: int(7.5) + 3
```

```
Out[ ]: 10
```

```
In [ ]: bool(0)
```

```
Out[ ]: False
```

```
In [ ]: #Auto typecasting  
True + 3 + int(4.5)
```

```
Out[ ]: 8
```

```
In [ ]: str(3) + 'vivek'
```

```
Out[ ]: '3vivek'
```

```
In [ ]: #Manual / forced type casting
```

```
4 + float('9.5')
```

Out[]: 13.5

```
In [ ]: int('7') + 5
```

Out[]: 12

```
In [ ]: a = 3.4  
type(a)
```

Out[]: float

```
In [ ]: #forced / manual typecasting  
int(a)
```

Out[]: 3

```
In [ ]: a = 3  
b = 4.5  
print(type(a))  
print(type(b))  
  
<class 'int'>  
<class 'float'>
```

```
In [ ]: a + int(b)
```

Out[]: 7

```
In [ ]: 3 + int('4')
```

Out[]: 7

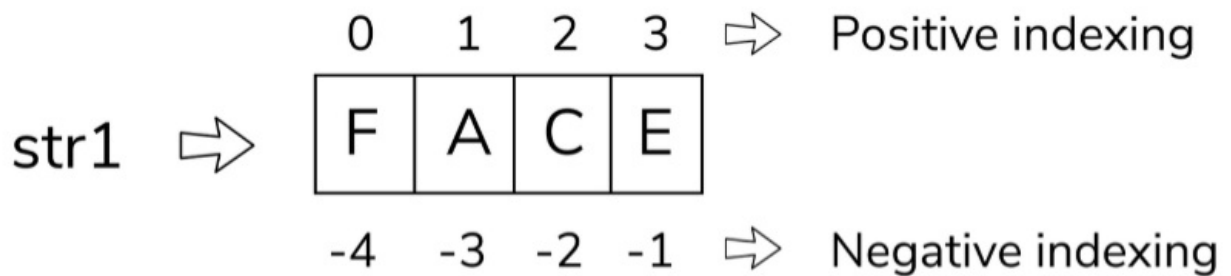
```
In [ ]: # True -> 1, False -> 0  
False + 4
```

Out[]: 4

```
In [ ]: int(3.4 + bool(-20)+int(8.4)) #did you get this
```

Out[]: 12

Slicing



str1[1:3] = AC

str1[-3:-1] = AC

Python slice() Function

A slice object is used to specify how to slice a sequence. You can specify where to start the slicing, and where to end. You can also specify the step, which allows you to e.g. slice only every other item.

```
In [ ]: a = "I am a Data Scientist" #indexing start from 0 from I & we count space as well so, index of a is 2, did you u
a      # "I=0", "space=1", "a=2", "m=3" & so on.
      # "t=-1", "s=-2" & so on in case of reverse indexing
```

```
Out[ ]: 'I am a Data Scientist'
```

```
In [ ]: a[2:4] #it will print a leetr which is on index 2 & 3 excluding index 4
```

```
Out[ ]: 'am'
```

```
In [ ]: a[-9:] #if we see index in reverse direction will start from -1
      #so, [-9:] from index -9 it will print all letter including a word at index 9
```

```
Out[ ]: 'Scientist'
```

```
In [ ]: # : -> slicing operator
a[7:] #starting from index 7(including) it will print till end
```

```
Out[ ]: 'Data Scientist'
```

```
In [ ]: a[:7] #print everything excluding the letters starting from index 7
```

```
Out[ ]: 'I am a '
```

Math Operators

Python Arithmetic Operators

Arithmetic operators are used with numeric values to perform common mathematical operations:

```
In [ ]: print(10 + 4) # add (returns 14)
print(10 - 4) # subtract (returns 6)
print(10 * 4) # multiply (returns 40)
print(10**4) # exponent (returns 10000)
print(10 / 4) # divide (returns 2.5)
print(5 % 4) # modulo (returns 1) - also known as the remainder

14
6
40
10000
2.5
1
```

Logical / Comparison Operators

Logical operators are used to combine conditional statements while Comparison operators are used to compare two values.

```
In [ ]: # comparisons (these return True)
print(5 > 3 )
print(5 >= 3)
print(5 != 5)
print(5 == 5) # boolean operations (these return True)

# evaluation order: not, and, or

True
True
False
True
```

Logica Operators

T and T --> T

T and F --> F

F and T --> F

F and F --> F

AND is only True if all are True

T or T --> T

T or F --> T

F or T --> T

F or F --> F

OR is only False if all are False

```
In [ ]: # Logical operators and or
(5>3) or (10>12)
```

Out[]: True

```
In [ ]: print(5 > 3 and 6 < 3 )
print(5 > 3 or 5 < 3 )
```

False

True

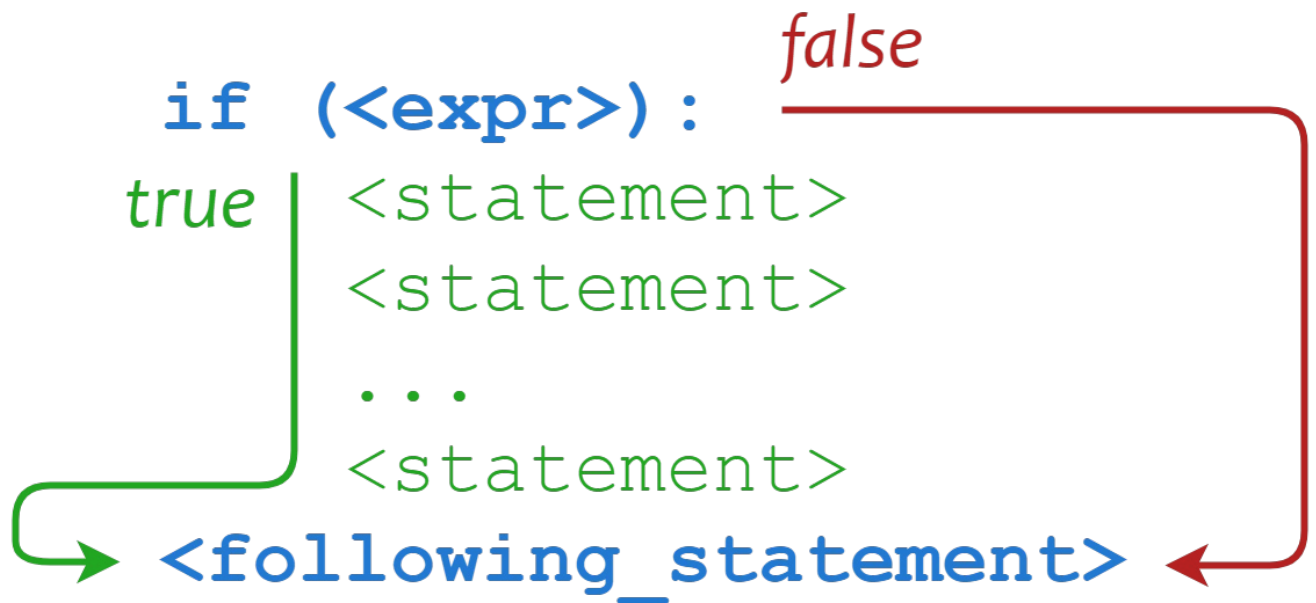
```
In [ ]: True == bool(-18)
```

```
Out[ ]: True
```

```
In [ ]: (5 >= 3 or 6 > 100) and (True == bool(23))
```

```
Out[ ]: True
```

Conditional Statement



```
In [ ]: x = 2

if (x>0):
    print("Positive number")
    print("In the IF block")
else:
    print("Negative number")
    print("In the else block")
```

```
Positive number
In the IF block
```

```
In [ ]: x=5
if (x>0):
    print("X is a positive")
    print("I m if True Block")
else:
    print("X is a Negative")
    print("I m if Else/False Block")

print("I am out of IF-ELSE block")
```

```
X is a positive
I m if True Block
I am out of IF-ELSE block
```

```
In [ ]: if 5 < 3:
    print("I am in if block")
    print("So the statement is TRUE")
```

```
else:
    print("I am in ELSE block")

print("I am anyway printed, out of IF")
```

I am in ELSE block
I am anyway printed, out of IF

```
In [ ]: if (5<3) :
        print("True")
        print("another statement")
    else :
        print("False")
        print("another else st")
    print("This prints anyway")
```

False
another else st
This prints anyway

More examples

```
In [ ]: x=12
        if (x>10) :
            print("This is True or IF block")
            print("I am still in IF")
        else :
            print("This is else block")

        print("\n ---- \n I am out of IF block")
```

This is True or IF block
I am still in IF

I am out of IF block

```
In [ ]: if (5<3):
        print("This is IF block")
    else :
        print("This is Else Block")
```

This is Else Block

```
In [ ]: if (5<3) :
        print("True block statement 1")
        print("True block statement 2")
        print("True block statement 3")
    else:
        print("False block")
```

False block

```
In [ ]: x = 0
        if (x > 0) :
            print("X is Positive")

        elif (x<0):
            print("X is Negative")
        else:
            print("X is ZERO")
```

X is ZERO

```
In [ ]: x=-100
```

```

if ((x>0) or (x==-100)):
    print("X is positive Value or -100")
    print("I am if loop")
elif (x<0):
    print("I am in else if block")
    print("X is negative")
else:
    print("X is Zero")

print("I am out of IF loop")

```

X is positive Value or -100
I am if loop
I am out of IF loop

In []:

```

x = 6

if x%2 == 0 :
    print(x, " is even number")
    print("hello..")
else :
    print(x, " is ODD number")

print("this is out of IF else block")

```

6 is even number
hello..
this is out of IF else block

In []:

```

x = -20
# if/elif/else statement
if x > 0:
    print('positive')
    print('hello')
elif x == 0:
    print('zero')
else:
    print('negative')
print("I am out of IF block")

```

negative
I am out of IF block

In []:

```

# single-line if statement (sometimes discouraged)
x=5
if x > 0: print('positive')

```

positive

Variables and Strings

Variables are used to store values. A string is a series of characters, surrounded by single or double quotes.

In []:

```

# Hello world
print('Hello world')

```

Hello world

In []:

```

# Hello world with a variable
msg = "Hello world!"
print(msg)

```

Hello world!

In []:

```
# Concatenation (combining strings)
first_name = 'albert'
last_name = 'einstein'
full_name = first_name + ' ' + last_name
print(full_name)
```

albert einstein

```
a='hi' # assigning the strig 'hi' to variable a
a
```

'hi'

```
strn=""Hi,
"How are you" "" # assigning multi_line string to varibale strn
strn
```

'Hi,\n"How are you" '

```
len(a) # Return the number of characters in a
```

2

```
c='GoodMorning'
c.startswith("Go") #Test whether c starts with the substring "Go"
```

True

```
c.endswith("gn") # Test whether c ends with the substring "gn"
```

False

```
c.replace("i","y") # Return a new string basedon c with all occurances of "i" replaced with "y"
```

'GoodMornyng'

```
strn.split(" ") # Split the string strn into a list of strings, separating on the character " " and return that
```

['Hi,\n"How', 'are', 'you"', '']

```
"{} plus {} plus {} is {}".format(1,2,4,7) # Return the string with the values 3, 1, and 4 inserted
```

'1 plus 2 plus 4 is 7'

```
c.lower() # Returns a lowercase version of c
```

'goodmorning'

```
c.upper() # Returns a uppercase version of c
```

'GOODMORNING'

```
In [ ]: c.title() # Returns c with the first letter of every word capitalized
```

```
Out[ ]: 'Goodmorning'
```

```
In [ ]: strn.splitlines() # Returns a list by splitting the string on any newline characters.
```

```
Out[ ]: ['Hi,', '"How are you" ']
```

```
In [ ]: c[:5] # Returns the first 5 characters of s
```

```
Out[ ]: 'GoodM'
```

```
In [ ]: "Get" + "Lost" # Returns "GetLost"
```

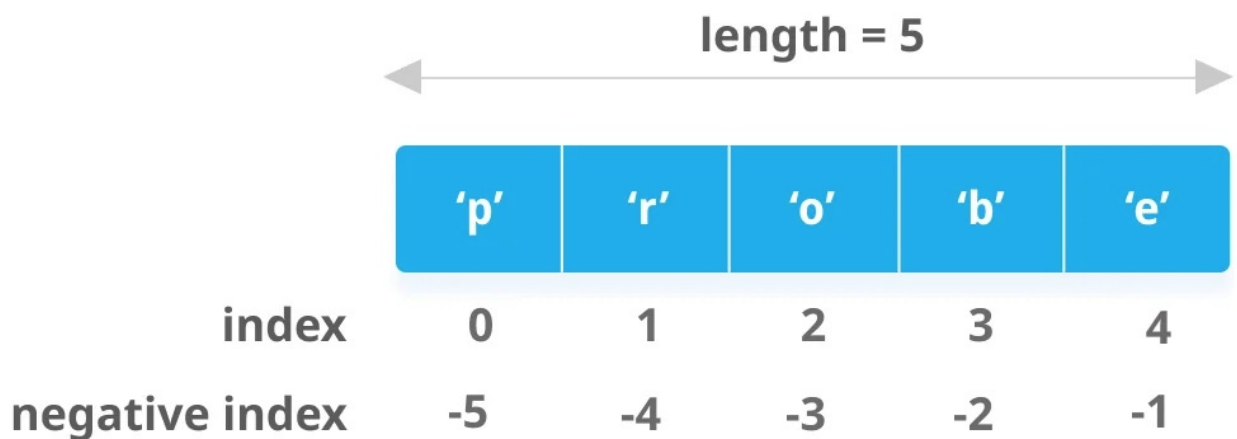
```
Out[ ]: 'GetLost'
```

```
In [ ]: "Mor" in c # Returns True if the substring "end" is found in c
```

```
Out[ ]: True
```

Lists

A list stores a series of items in a particular order. You access items using an index, or within a loop.



```
In [ ]: # Make a list
bikes = ['trek', 'redline', 'giant']
```

```
In [ ]: # Get the first item in a list
first_bike = bikes[0]
first_bike
```

```
Out[ ]: 'trek'
```

```
In [ ]: # Get the last item in a list
```

```
last_bike = bikes[-1]
last_bike
```

Out[]: 'giant'

```
In [ ]: # Looping through a list
for bike in bikes:
    print(bike)
```

```
trek
redline
giant
```

```
In [ ]: # Adding items to a list
bikes = []
bikes.append('trek')
bikes.append('redline')
bikes.append('giant')
print(bikes)
```

```
['trek', 'redline', 'giant']
```

```
In [ ]: # Making numerical lists
squares = []
for x in range(1, 11):
    squares.append(x**2)
squares
```

Out[]: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

```
In [ ]: # List comprehensions
squares = [x**2 for x in range(1, 11)]
squares
```

Out[]: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

```
In [ ]: # Slicing a list
finishers = ['sam', 'bob', 'ada', 'bea']
first_two = finishers[:2]
first_two
```

Out[]: ['sam', 'bob']

```
In [ ]: # Copying a list
copy_of_bikes = bikes[:]
copy_of_bikes
```

Out[]: ['trek', 'redline', 'giant']

```
In [ ]: z=[100,20,30,45,68,54]
# Return the first value in the list z
z[0]
```

Out[]: 100

```
In [ ]: # Return the last value in the list z
z[-1]
```

Out[]: 54

```
In [ ]: # Return a slice (list) containing the fourth and fifth values of z
        z[3:5]
```

```
Out[ ]: [45, 68]
```

```
In [ ]: len(z) # Return the number of elements in z
```

```
Out[ ]: 6
```

```
In [ ]: Sum=sum(z) # Return the sum of the values of z
        print(Sum)
```

```
317
```

```
In [ ]: min(z) # Return the minimum value from a
```

```
Out[ ]: 20
```

```
In [ ]: max(z) # Return the maximum value from a
```

```
Out[ ]: 100
```

```
In [ ]: z.append(21) # Append the value 21 to the end of a
        z
```

```
Out[ ]: [100, 20, 30, 45, 68, 54, 21]
```

```
In [ ]: z.sort() # Sort the items of a in ascending order
        z
```

```
Out[ ]: [20, 21, 30, 45, 54, 68, 100]
```

```
In [ ]: z.sort(reverse=True) # Sort the items of a in descending order
        z
```

```
Out[ ]: [100, 68, 54, 45, 30, 21, 20]
```

```
In [ ]: " ".join(["A", "B", "C", "D"]) # Converts the list["A", "B", "C", "D"] into the string "A B C D"
```

```
Out[ ]: 'A B C D'
```

```
In [ ]: z.pop(3) # Returns the fourth item from a and deletes it from the list
```

```
Out[ ]: 45
```

```
In [ ]: z # 45 got deleted
```

```
Out[ ]: [100, 68, 54, 30, 21, 20]
```

```
In [ ]: z.remove(30) # Removes the first item in a that is equal to 30
```

```
In [ ]: z # 30 got removed
```

```
Out[ ]: [100, 68, 54, 21, 20]
```

```
In [ ]: z.reverse() # Reverses the order of the items in a  
z
```

```
Out[ ]: [20, 21, 54, 68, 100]
```

```
In [ ]: z[1::2] # Returns every second item from a, commencing from the 1st item
```

```
Out[ ]: [21, 68]
```

```
In [ ]: z[-5:] # Returns the last 5 items from a specific axis
```

```
Out[ ]: [20, 21, 54, 68, 100]
```

```
In [ ]: z[0]=12 # assigning a value to required with its index  
z
```

```
Out[ ]: [12, 21, 54, 68, 100]
```

Tuples

Tuples are similar to lists, but the items in a tuple can't be modified.

```
t = (1, 2, 'Python', tuple(), (42, 'hi'))
```

t[0] t[1] t[2] t[3] t[4]

```
In [ ]: # Creating a non empty tuple  
tuple='java','anadroid','CSS','HTML'  
print(tuple)
```

```
('java', 'anadroid', 'CSS', 'HTML')
```

```
In [ ]: # Concating 2 tuples  
tuple_1='Android','java','HTML'  
tuple_2=5,8,6,9  
print(tuple_1 + tuple_2)
```

```
('Android', 'java', 'HTML', 5, 8, 6, 9)
```

```
In [ ]: # repetition  
tup=('Hello,')*5
```

```
print(tup)

('Hello', 'Hello', 'Hello', 'Hello', 'Hello')
```

```
In [ ]: # Nesting of Tuples
        tup_1=('Python','Java','CSS','HTML')
        tup_2=(1,5,8,6,7,3)
        tup_3=(tup_1,tup_2)
        print(tup_3)

(('Python', 'Java', 'CSS', 'HTML'), (1, 5, 8, 6, 7, 3))
```

```
In [ ]: # Slicing the tuples
        a=(8,3,6,9,45,78,69,12,36)
        print(a[1:])
        print(a[::-1])
        print(a[2:4])

(3, 6, 9, 45, 78, 69, 12, 36)
(36, 12, 69, 78, 45, 9, 6, 3, 8)
(6, 9)
```

```
In [ ]: # lenght of tuple
        t=('A','B','C','D')
        print(len(t))

4
```

```
In [ ]: # Tuples in loop
        A = ('Hello_World',)
        n = 10 #Number of time loop runs
        for i in range(int(n)):
            tup = (A,)
            print(tup)

(('Hello_World',),)
(('Hello_World',),)
(('Hello_World',),)
(('Hello_World',),)
(('Hello_World',),)
(('Hello_World',),)
(('Hello_World',),)
(('Hello_World',),)
(('Hello_World',),)
(('Hello_World',),)
```

```
In [ ]: # create a tuple
        digits1 = (0, 1, 'two',0,1,1,1) # create a tuple directly
        # examine a tuple
        print(digits1.count(1)) # counts the number of instances of that value (0) digits.index(1) # returns the index
        len(digits1)
```

4

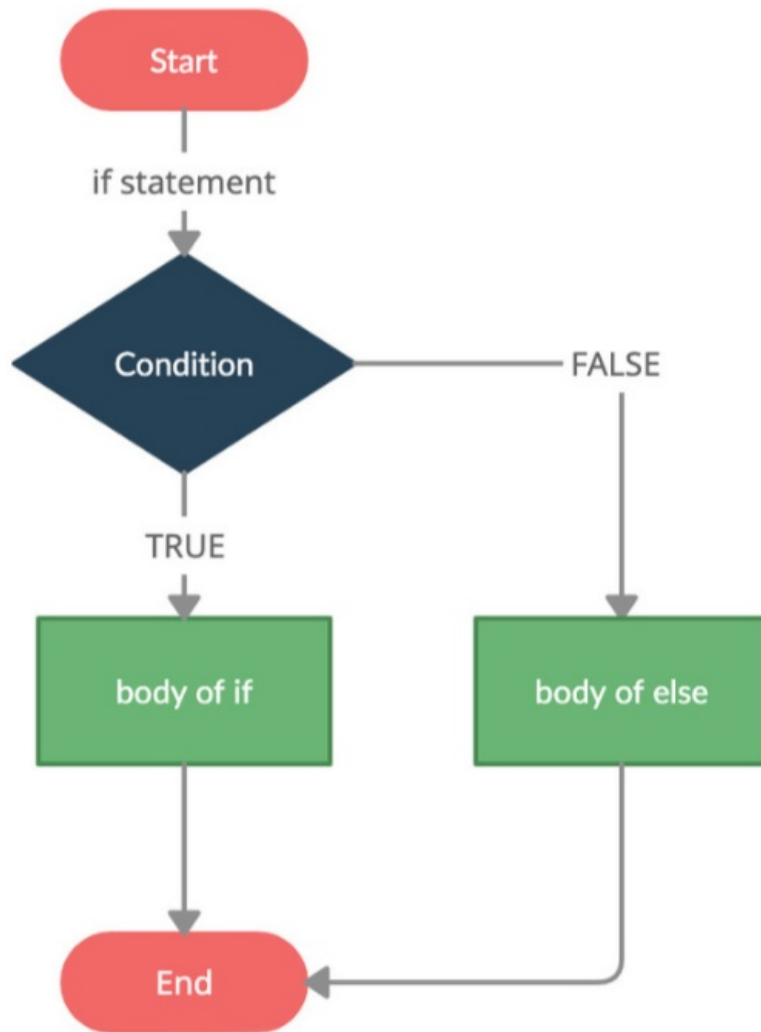
```
Out[ ]: 7
```

```
In [ ]: # using min(),max()
        tuple_A=(5,4,3,2,1,8,7)
        tuple_B=('Hello','Hi','Bye','Good Morning')
        print('max value in tuple_A and B:'+ str(max(tuple_A)) + ' ' + str(max(tuple_B)))
        print('min value in tuple_A and B:'+ str(min(tuple_A))+ ' ' + str(min(tuple_B)))

max value in tuple_A and B:8,Hi
min value in tuple_A and B:1,Bye
```

If Statements

If statements are used to test for particular conditions and respond appropriately.



```
In [ ]: # if
mark = 10
if (mark > 15):
    print ("mark is less than 15")
print ("I am Not in if")
```

I am Not in if

```
In [ ]: # if-else
mark = 20;
if (mark < 15):
    print ("mark is less than 15")
    print ("i'm in if Block")
else:
    print ("mark is greater than 15")
    print ("i'm in else Block")
print ("i'm not in if and not in else Block")
```

mark is greater than 15
i'm in else Block
i'm not in if and not in else Block

```
In [ ]: # nested-if
mark = 10
if (mark == 10):
    # First if statement
    if (mark < 15):
        print ("mark is smaller than 15")
```

```
# Nested - if statement  
# Will only be executed if statement above  
# it is true  
if (mark < 12):  
    print ("mark is smaller than 12 too")  
else:  
    print ("mark is greater than 15")
```

mark is smaller than 15
mark is smaller than 12 too

```
In [ ]: # if-elif-else ladder  
mark = 20  
if (mark == 10):  
    print ("mark is 10")  
elif (mark == 15):  
    print ("mark is 15")  
elif (mark == 20):  
    print ("mark is 20")  
else:  
    print ("mark is not present")
```

mark is 20

```
In [ ]: # A simple if test  
age=20  
if age >= 18:  
    print("You can vote!")
```

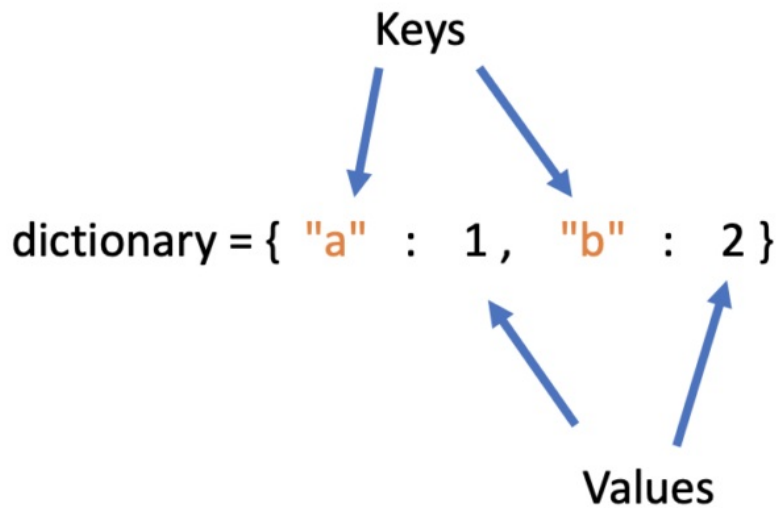
You can vote!

```
In [ ]: # if-elif-else statements  
age=10  
if age <=4:  
    print('ticket_price = 0')  
elif age < 18:  
    print('ticket_price = 10')  
else:  
    print('ticket_price = 15')
```

ticket_price = 10

Dictionaries

Dictionaries store connections between pieces of information. Each item in a dictionary is a key-value pair.



```
In [ ]: # A simple dictionary
alien = {'color': 'green', 'points': 5}
```

```
In [ ]: # Accessing a value
print("The alien's color is " + alien['color'])
```

The alien's color is green

```
In [ ]: # Adding a new key-value pair
alien['x_position'] = 0
```

```
In [ ]: # Looping through all key-value pairs
fav_numbers = {'eric': 17, 'ever': 4}
for name, number in fav_numbers.items():
    print(name + ' loves ' + str(number))
```

eric loves 17
ever loves 4

```
In [ ]: # Looping through all keys
fav_numbers = {'eric': 17, 'ever': 4}
for name in fav_numbers.keys():
    print(name + ' loves a number')
```

eric loves a number
ever loves a number

```
In [ ]: # Looping through all the values
fav_numbers = {'eric': 17, 'ever': 4}
for number in fav_numbers.values():
    print(str(number) + ' is a favorite')
```

17 is a favorite
4 is a favorite

```
In [ ]: # creating a dict with NY,IN,UK as key and their full form as values
dict = {"NY": "New_York", "IN": "India", "UK": "United_Kingdom"}
```

```
In [ ]: dict.keys() # Return a list of the keys from dict
```

```
Out[ ]: dict_keys(['NY', 'IN', 'UK'])
```

```
In [ ]: dict.values() # Return a list of the values from dict
```

```
Out[ ]: dict_values(['New_York', 'India', 'United_Kingdom'])
```

```
In [ ]: dict.items() # Return a list of (key, value) from dict
```

```
Out[ ]: dict_items([('NY', 'New_York'), ('IN', 'India'), ('UK', 'United_Kingdom')])
```

```
In [ ]: max(dict, key=dict.get) # Return the key that corresponds to the largest value in dict
```

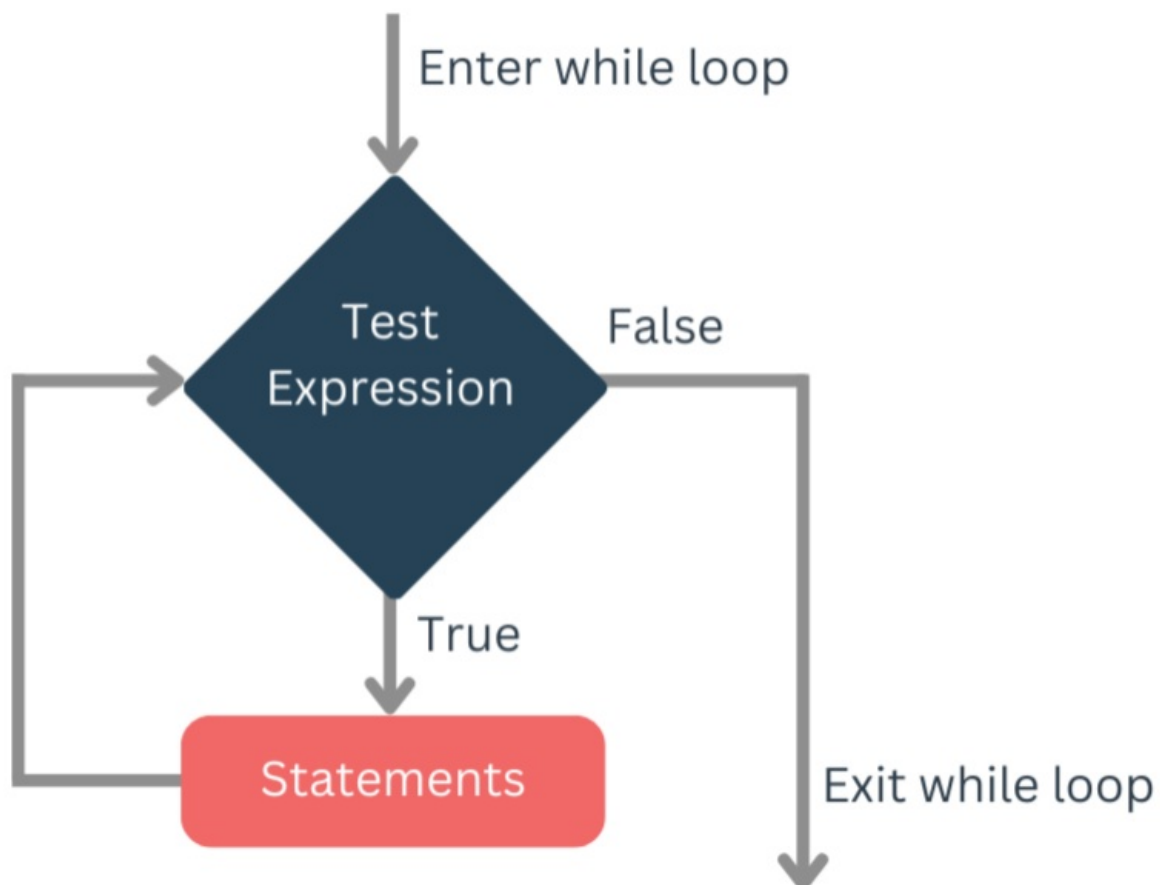
```
Out[ ]: 'UK'
```

```
In [ ]: min(dict, key=dict.get) # Return the key that corresponds to the smallest value in dict
```

```
Out[ ]: 'IN'
```

While Loop

A while loop repeats a block of code as long as a certain condition is true.



```
In [ ]: # A simple while loop
current_value = 1
while current_value <= 5:
```

```
print(current_value)
current_value += 1
```

1
2
3
4
5

```
In [ ]: # Letting the user choose when to quit
msg = ''
while msg != 'quit':
    msg = input("What's your message? ")
    print(msg)
```

What's your message? Hi everyone
Hi everyone
What's your message? quit
quit

```
In [ ]: # Single statement while block
count = 0
while (count < 5): count += 1; print("Hello")
```

Hello
Hello
Hello
Hello
Hello

```
In [ ]: # loop control statement
# Continue Statement: It returns the control to the beginning of the loop
# Prints all letters except 'w' and 'r'
i = 0
a = 'doyourwork'

while i < len(a):
    if a[i] == 'w' or a[i] == 'r':
        i += 1
        continue
    print('Current Letter :', a[i])
    i += 1
```

Current Letter : d
Current Letter : o
Current Letter : y
Current Letter : o
Current Letter : u
Current Letter : r
Current Letter : o
Current Letter : r
Current Letter : k

```
In [ ]: # break the loop as soon it sees 'e' or 'g'
i = 0
a = 'HappyLearning'

while i < len(a):
    if a[i] == 'e' or a[i] == 'g':
        i += 1
        break
    print('Current Letter :', a[i])
    i += 1
```

Current Letter : H
Current Letter : a
Current Letter : p
Current Letter : p
Current Letter : y
Current Letter : L

```
In [ ]: # break the loop as soon it sees 'm' or 'z'
i = 0
a = 'yuweffuygmewedwz'

while i < len(a):
    if a[i] == 'm' or a[i] == 'z':
        i += 1
        break
    print('Current Letter :', a[i])
    i += 1
```

Current Letter : y
Current Letter : u
Current Letter : w
Current Letter : e
Current Letter : f
Current Letter : f
Current Letter : u
Current Letter : y
Current Letter : g

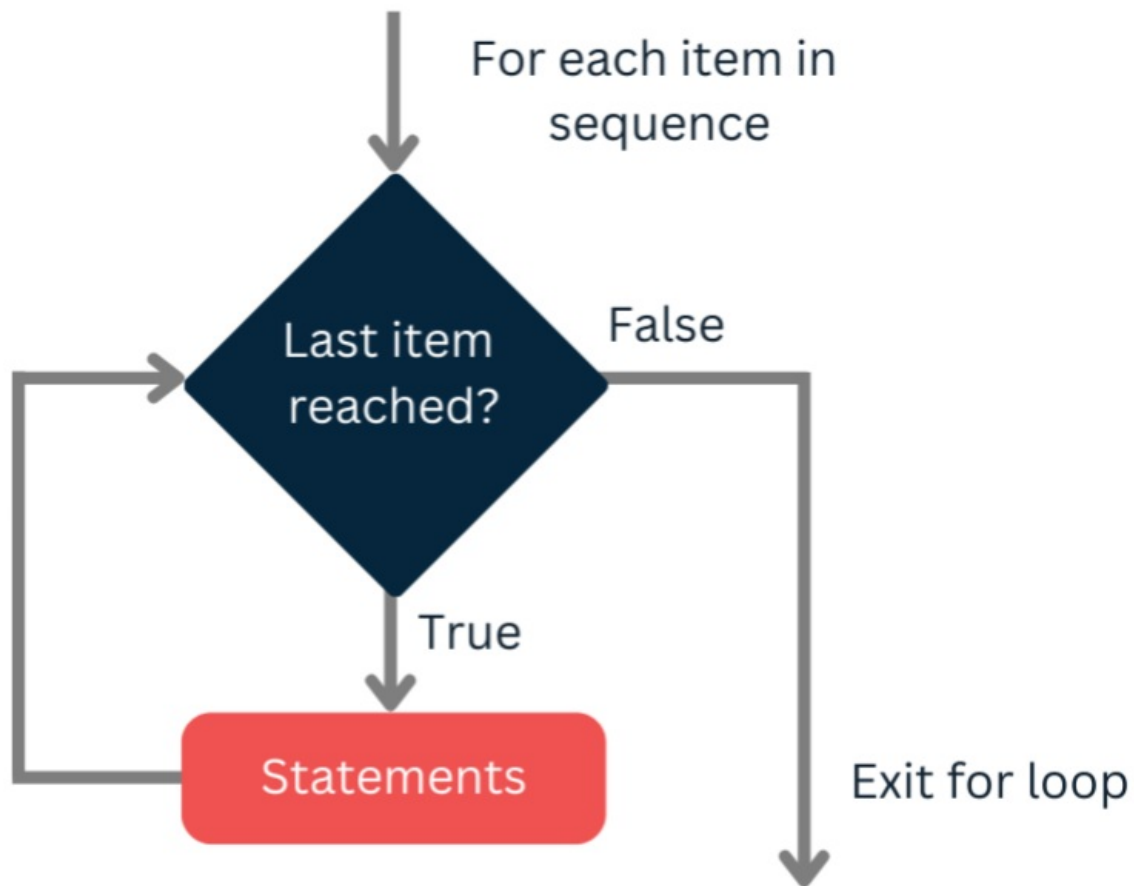
```
In [ ]: # while-else loop
i = 0
while i < 8:
    i += 1
    print(i)
else: # Executed because no break in for
    print("No Break\n")

i = 0
while i < 8:
    i += 1
    print(i)
    break
else: # Not executed as there is a break
    print("No Break")
```

1
2
3
4
5
6
7
8
No Break

1

For Loop



A for loop is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set, or a string).

```
In [ ]: # Program to print squares of all numbers present in a list

# List of integer numbers
numbers = [1, 2, 4, 6, 11, 20]

# variable to store the square of each num temporary
sq = 0

# iterating over the given list
for val in numbers:
    # calculating square of each number
    sq = val * val
    # displaying the squares
    print(sq)
```

```
1
4
16
36
121
400
```

```
In [ ]: # Python for loop example using range() function
# Program to print the sum of first 5 natural numbers

# variable to store the sum
sum = 0

# iterating over natural numbers using range()
for val in range(1, 6):
    # calculating sum
    sum = sum + val

# displaying sum of first 5 natural numbers
print(sum)
```

15

```
In [ ]: # For loop with else block
for val in range(5):
    print(val)
else:
    print("The loop has completed execution")
```

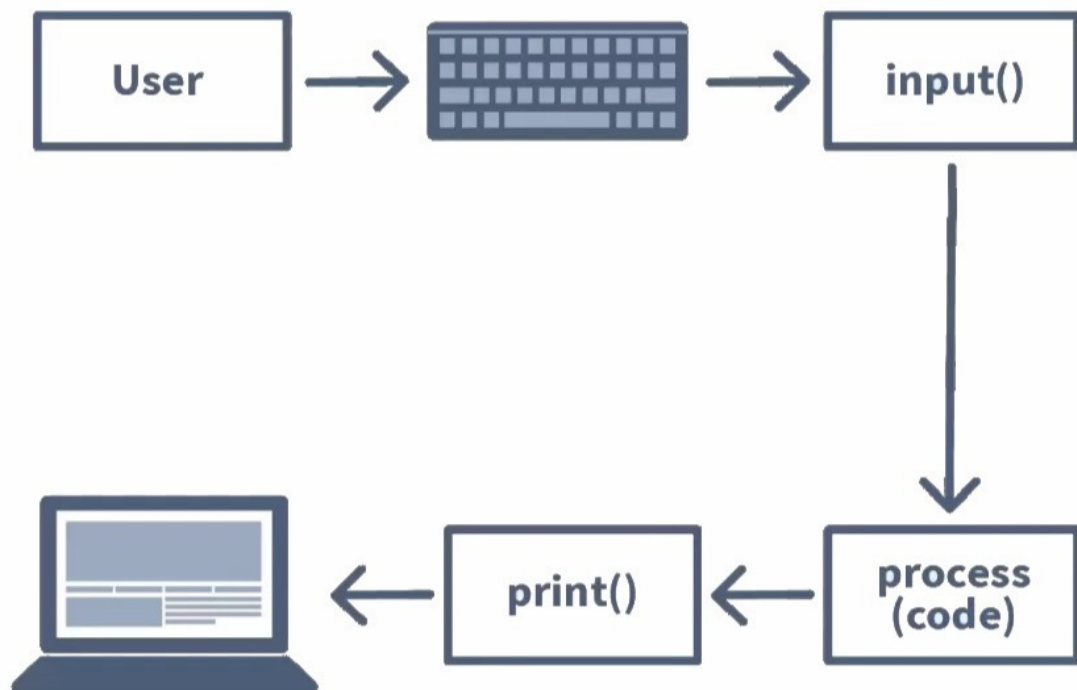
0
1
2
3
4
The loop has completed execution

```
In [ ]: #Nested For loop in Python
for num1 in range(3):
    for num2 in range(10, 14):
        print(num1, ",", num2)
```

0 , 10
0 , 11
0 , 12
0 , 13
1 , 10
1 , 11
1 , 12
1 , 13
2 , 10
2 , 11
2 , 12
2 , 13

User Inputs

Your programs can prompt the user for input. All input is stored as a string.



```
In [ ]: # Prompting for a value
name = input("What's your name? ")
print("Hello, " + name + "!")
```

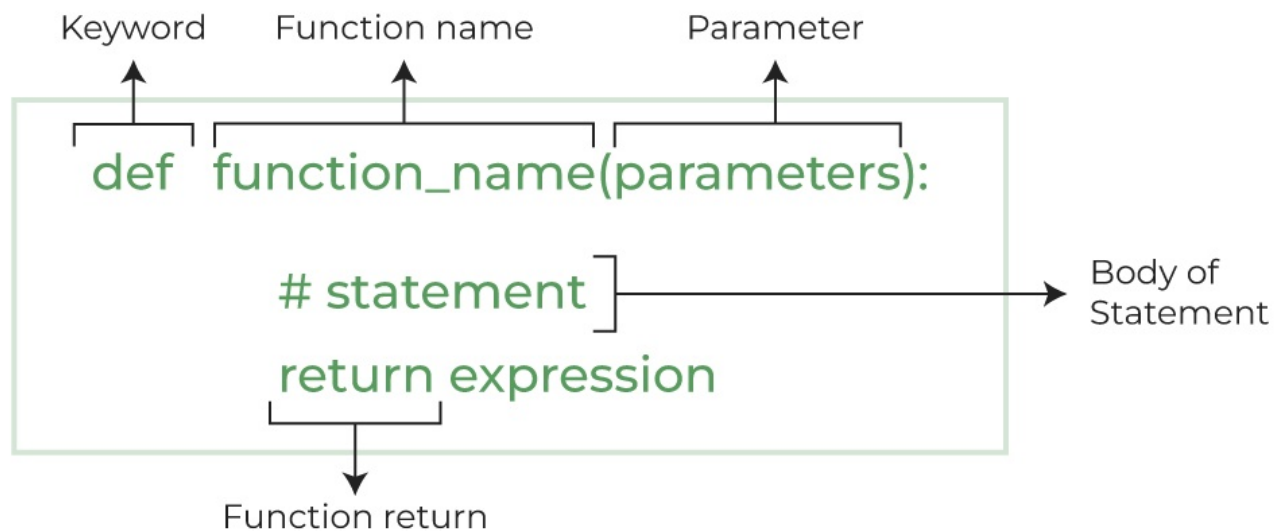
What's your name? Vivek
Hello, Vivek!

```
In [ ]: # Prompting for numerical input
```

```
age = input("How old are you? ")
age = int(age)
pi = input("What's the value of pi? ")
pi = float(pi)
```

How old are you? 22
What's the value of pi? 3.14

Functions



Functions are named blocks of code, designed to do one specific job. Information passed to a function is called an argument, and information received by a function is called a parameter.

```
In [ ]: #Making a function
def greet_user():
    """Display a simple greeting."""
    print("Hello!")
greet_user()
```

Hello!

```
In [ ]: #Passing a single argument
def greet_user(username):
    """Display a simple greeting."""
    print("Hello, " + username + "!")
greet_user('jesse')
greet_user('diana')
greet_user('brandon')
```

Hello, jesse!
Hello, diana!
Hello, brandon!

```
In [ ]: #A simple function
def greet_user():
    """Display a simple greeting."""
    print("Hello!")
greet_user()
```

Hello!

```
In [ ]: # Passing an argument
def greet_user(username):
    """Display a personalized greeting."""
    print("Hello, " + username + "!")
greet_user('jesse')
```

Hello, jesse!

```
In [ ]: #Default values for parameters
def make_pizza(topping='bacon'):
    """Make a single-topping pizza."""
    print("Have a " + topping + " pizza!")
make_pizza()
make_pizza('pepperoni')
```

Have a bacon pizza!
Have a pepperoni pizza!

```
In [ ]: # Returning a value
def add_numbers(x, y):
    """Add two numbers and return the sum."""
    return x + y
sum = add_numbers(3, 5)
print(sum)
```

8

```
In [ ]: # Using positional arguments
def describe_pet(animal, name):
    """Display information about a pet."""
    print("\nI have a " + animal + ".")
    print("Its name is " + name + ".")
describe_pet('hamster', 'harry')
describe_pet('dog', 'willie')
# Using keyword arguments
def describe_pet(animal, name):
    """Display information about a pet."""
    print("\nI have a " + animal + ".")
    print("Its name is " + name + ".")
describe_pet(animal='hamster', name='harry')
describe_pet(name='willie', animal='dog')
```

I have a hamster.
Its name is harry.

I have a dog.
Its name is willie.

I have a hamster.
Its name is harry.

I have a dog.
Its name is willie.

```
In [ ]: # Using a default value
def describe_pet(name, animal='dog'):
    """Display information about a pet."""
    print("\nI have a " + animal + ".")
    print("Its name is " + name + ".")
describe_pet('harry', 'hamster')
describe_pet('willie')
# Using None to make an argument optional
def describe_pet(animal, name=None):
    """Display information about a pet."""
    print("\nI have a " + animal + ".")
    if name:
        print("Its name is " + name + ".")
describe_pet('hamster', 'harry')
describe_pet('snake')
```

I have a hamster.
Its name is harry.

I have a dog.
Its name is willie.

I have a hamster.
Its name is harry.

I have a snake.

```
In [ ]: # Using a default value
def describe_pet(name, animal='dog'):
    """Display information about a pet."""
    print("\nI have a " + animal + ".")
    print("Its name is " + name + ".")
describe_pet('harry', 'hamster')
describe_pet('willie')
# Using None to make an argument optional
def describe_pet(animal, name=None):
    """Display information about a pet."""
    print("\nI have a " + animal + ".")
    if name:
        print("Its name is " + name + ".")
describe_pet('hamster', 'harry')
describe_pet('snake')
```

I have a hamster.
Its name is harry.

I have a dog.
Its name is willie.

I have a hamster.
Its name is harry.

I have a snake.

```
In [ ]: #Passing a list as an argument
def greet_users(names):
    """Print a simple greeting to everyone."""
    for name in names:
        msg = "Hello, " + name + "!"
        print(msg)
usernames = ['hannah', 'ty', 'margot']
greet_users(usernames)

#Allowing a function to modify a list
#The following example sends a list of models to a function for
#printing. The original list is emptied, and the second list is filled.
def print_models(unprinted, printed):
    """3d print a set of models."""
    while unprinted:
        current_model = unprinted.pop()
        print("Printing " + current_model)
        printed.append(current_model)
# Store some unprinted designs,
# and print each of them.
unprinted = ['phone case', 'pendant', 'ring']
printed = []
print_models(unprinted, printed)
print("\nUnprinted:", unprinted)
print("Printed:", printed)

#Preventing a function from modifying a list
#The following example is the same as the previous one, except the
#original list is unchanged after calling print_models().
def print_models(unprinted, printed):
    """3d print a set of models."""
    while unprinted:
        current_model = unprinted.pop()
        print("Printing " + current_model)
        printed.append(current_model)
# Store some unprinted designs,
# and print each of them.
original = ['phone case', 'pendant', 'ring']
printed = []
print_models(original[:], printed)
print("\nOriginal:", original)
print("Printed:", printed)
```

Hello, hannah!
Hello, ty!
Hello, margot!
Printing ring
Printing pendant
Printing phone case

Unprinted: []
Printed: ['ring', 'pendant', 'phone case']

Printing ring
Printing pendant
Printing phone case

Original: ['phone case', 'pendant', 'ring']
Printed: ['ring', 'pendant', 'phone case']

```
In [ ]: #Collecting an arbitrary number of arguments
def make_pizza(size, *toppings):
    """Make a pizza."""
    print("\nMaking a " + size + " pizza.")
    print("Toppings:")
    for topping in toppings:
        print("- " + topping)
# Make three pizzas with different toppings.
make_pizza('small', 'pepperoni')
make_pizza('large', 'bacon bits', 'pineapple')
make_pizza('medium', 'mushrooms', 'peppers', 'onions', 'extra cheese')

#Collecting an arbitrary number of keyword arguments
def build_profile(first, last, **user_info):
    """Build a user's profile dictionary."""
# Build a dict with the required keys.
    profile = {'first': first, 'last': last}
# Add any other keys and values.
    for key, value in user_info.items():
        profile[key] = value
    return profile
# Create two users with different kinds
# of information.
user_0 = build_profile('albert', 'einstein',
    location='princeton')
user_1 = build_profile('marie', 'curie',
    location='paris', field='chemistry')
print(user_0)
print(user_1)
```

Making a small pizza.

Toppings:
- pepperoni

Making a large pizza.

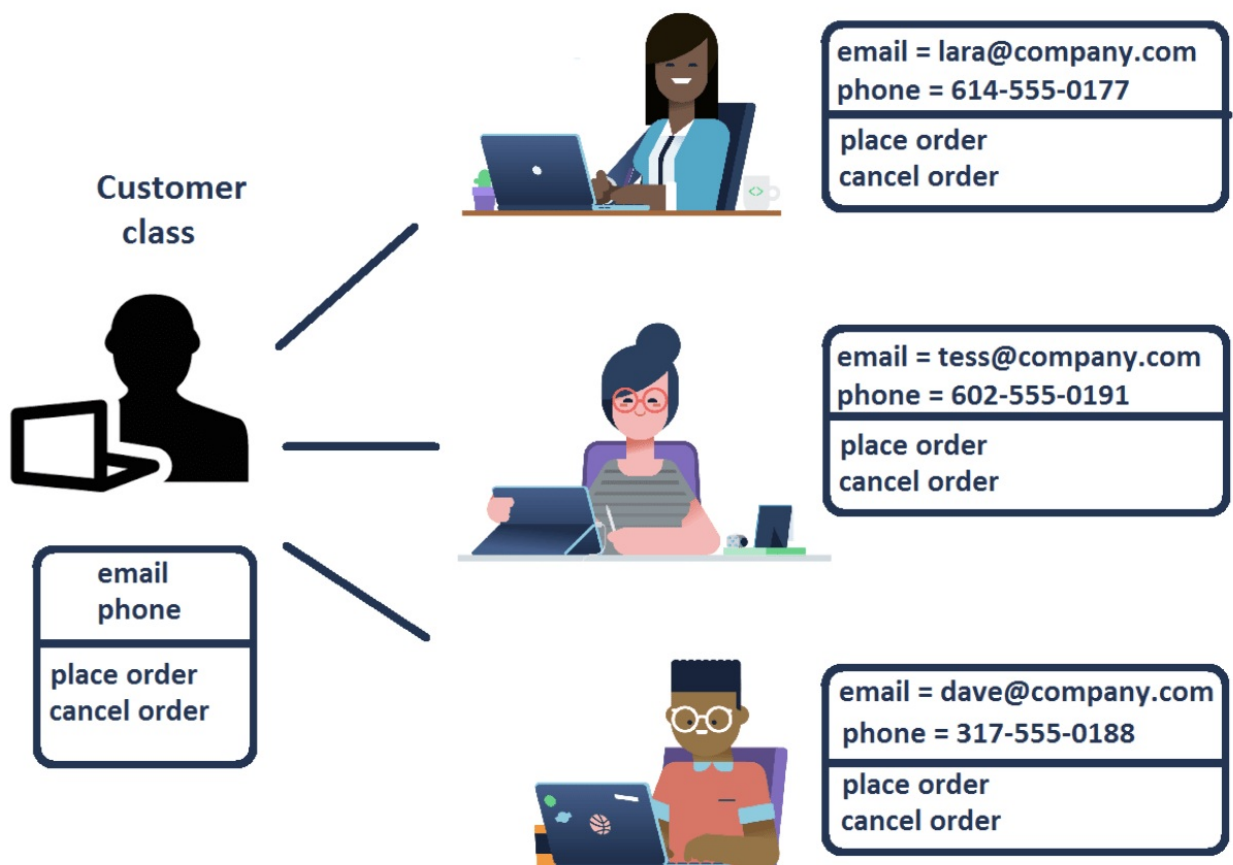
Toppings:
- bacon bits
- pineapple

Making a medium pizza.

Toppings:
- mushrooms
- peppers
- onions
- extra cheese

{'first': 'albert', 'last': 'einstein', 'location': 'princeton'}
{'first': 'marie', 'last': 'curie', 'location': 'paris', 'field': 'chemistry'}

Classes



A class defines the behavior of an object and the kind of information an object can store. The information in a class is stored in attributes, and functions that belong to a class are called methods. A child class inherits the attributes and methods from its parent class.

```
In [ ]: # Creating a dog class
class Dog():
    """Represent a dog."""
    def __init__(self, name):
        """Initialize dog object."""
        self.name = name
    def sit(self):
        """Simulate sitting."""
        print(self.name + " is sitting.")
my_dog = Dog('Peso')
print(my_dog.name + " is a great dog!")
my_dog.sit()
```

```
Peso is a great dog!
Peso is sitting.
```

```
In [ ]: # Inheritance
class SARDog(Dog):
    """Represent a search dog."""
    def __init__(self, name):
        """Initialize the sardog."""
        super().__init__(name)
    def search(self):
        """Simulate searching."""
        print(self.name + " is searching.")
my_dog = SARDog('Willie')
print(my_dog.name + " is a search dog.")
my_dog.sit()
my_dog.search()
```

```
Willie is a search dog.
Willie is sitting.
Willie is searching.
```

```
In [ ]: #The Car class
class Car():
```

```
"""A simple attempt to model a car."""
```

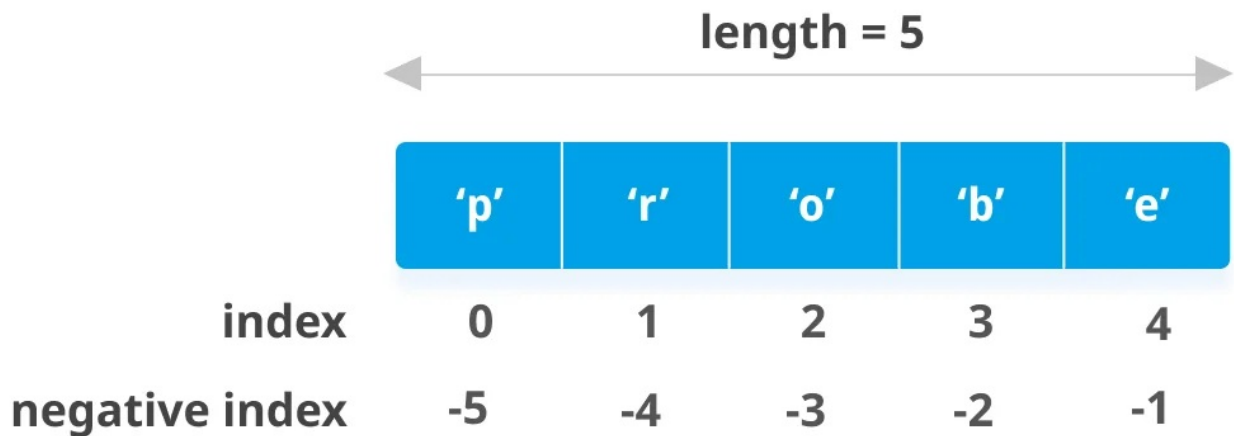
```
def __init__(self, make, model, year):  
    """Initialize car attributes."""  
    self.make = make  
    self.model = model  
    self.year = year  
# Fuel capacity and level in gallons.  
    self.fuel_capacity = 15  
    self.fuel_level = 0  
def fill_tank(self):  
    """Fill gas tank to capacity."""  
    self.fuel_level = self.fuel_capacity  
    print("Fuel tank is full.")  
  
def drive(self):  
    """Simulate driving."""  
    print("The car is moving.")
```

```
In [ ]: #Modifying an attribute directly  
my_new_car = Car('audi', 'a4', 2016)  
my_new_car.fuel_level = 5  
#Writing a method to update an attribute's value  
def update_fuel_level(self, new_level):  
    """Update the fuel level."""  
    if new_level <= self.fuel_capacity:  
        self.fuel_level = new_level  
    else:  
        print("The tank can't hold that much!")  
#Writing a method to increment an attribute's value  
def add_fuel(self, amount):  
    """Add fuel to the tank."""  
    if self.fuel_level + amount <= self.fuel_capacity:  
        self.fuel_level += amount  
        print("Added fuel.")  
    else:  
        print("The tank won't hold that much.")
```

```
In [ ]: #Creating an object from a class  
my_car = Car('audi', 'a4', 2016)  
#Accessing attribute values  
print(my_car.make)  
print(my_car.model)  
print(my_car.year)  
#Calling methods  
my_car.fill_tank()  
my_car.drive()  
#Creating multiple objects  
my_car = Car('audi', 'a4', 2016)  
my_old_car = Car('subaru', 'outback', 2013)  
my_truck = Car('toyota', 'tacoma', 2010)
```

```
audi  
a4  
2016  
Fuel tank is full.  
The car is moving.
```

Lists



Lists are used to store multiple items in a single variable.

Lists are one of 4 built-in data types in Python used to store collections of data, the other 3 are Tuple, Set, and Dictionary, all with different qualities and usage.

Lists are created using square brackets:

```
In [ ]: # Making a list
users = ['val', 'bob', 'mia', 'ron', 'ned']
```

```
In [ ]: # Getting the first element
first_user = users[0]
print(first_user)
#Getting the second element
second_user = users[1]
print(second_user)
#Getting the last element
newest_user = users[-1]
print(newest_user)
```

```
val
bob
ned
```

```
In [ ]: # Changing an element
users[0] = 'valerie'
users[-2] = 'ronald'
users
```

```
Out[ ]: ['valerie', 'bob', 'mia', 'ronald', 'ned']
```

```
In [ ]: # Adding an element to the end of the list
users.append('amy')
print(users)
# Starting with an empty list
users = []
users.append('val')
users.append('bob')
users.append('mia')
print(users)
# Inserting elements at a particular position
users.insert(0, 'joe')
users.insert(3, 'bea')
print(users)
```

```
['valerie', 'bob', 'mia', 'ronald', 'ned', 'amy']
['val', 'bob', 'mia']
['joe', 'val', 'bob', 'bea', 'mia']
```

```
In [ ]: # Deleting an element by its position
del users[-1]
print(users)
# Removing an item by its value
users.remove('bea')
print(users)

['joe', 'val', 'bob', 'bea']
['joe', 'val', 'bob']
```

```
In [ ]: # Pop the last item from a list
most_recent_user = users.pop()
print(most_recent_user)
# Pop the first item in a list
first_user = users.pop(0)
print(first_user)
```

bob
joe

```
In [ ]: #Find the length of a list
num_users = len(users)
print("We have " + str(num_users) + " users.")
```

We have 1 users.

```
In [ ]: # Sorting a list permanently
users.sort()
# Sorting a list permanently in reverse alphabetical order
users.sort(reverse=True)
# Sorting a list temporarily
print(sorted(users))
print(sorted(users, reverse=True))
# Reversing the order of a list
users.reverse()

['val']
['val']
```

```
In [ ]: # Printing all items in a list
for user in users:
    print(user)
#Printing a message for each item, and a separate message afterwards
for user in users:
    print("Welcome, " + user + "!")
print("Welcome, we're glad to see you all!")

val
Welcome, val!
Welcome, we're glad to see you all!
```

```
In [ ]: # Printing the numbers 0 to 1000
for number in range(1001):
    print(number)
#Printing the numbers 1 to 1000
for number in range(1, 1001):
    print(number)
#Making a list of numbers from 1 to a million
numbers = list(range(1, 1000001))
numbers

#It will give us output as numbers from 0 to 1000
```

```
In [ ]: # Finding the minimum value in a list
ages = [93, 99, 66, 17, 85, 1, 35, 82, 2, 77]
youngest = min(ages)
print(youngest)
# Finding the maximum value
ages = [93, 99, 66, 17, 85, 1, 35, 82, 2, 77]
```

```
oldest = max(ages)
print(oldest)
```

```
1
99
```

```
In [ ]: # Getting the first three items
finishers = ['kai', 'abe', 'ada', 'gus', 'zoe']
first_three = finishers[:3]
print(first_three)
# Getting the middle three items
middle_three = finishers[1:4]
print(middle_three)
# Getting the last three items
last_three = finishers[-3:]
print(last_three)
```

```
['kai', 'abe', 'ada']
['abe', 'ada', 'gus']
['ada', 'gus', 'zoe']
```

```
In [ ]: # Making a copy of a list
finishers = ['kai', 'abe', 'ada', 'gus', 'zoe']
copy_of_finishers = finishers[:]
print(copy_of_finishers)
```

```
['kai', 'abe', 'ada', 'gus', 'zoe']
```

```
In [ ]: # Using a loop to generate a list of square numbers
squares = []
for x in range(1, 11):
    square = x**2
    squares.append(square)
print(squares)
# Using a comprehension to generate a list of square numbers
squares = [x**2 for x in range(1, 11)]
print(squares)
# Using a loop to convert a list of names to upper case
names = ['kai', 'abe', 'ada', 'gus', 'zoe']
upper_names = []
for name in names:
    upper_names.append(name.upper())
print(upper_names)
# Using a comprehension to convert a list of names to upper case
names = ['kai', 'abe', 'ada', 'gus', 'zoe']
upper_names = [name.upper() for name in names]
print(upper_names)
```

```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
['KAI', 'ABE', 'ADA', 'GUS', 'ZOE']
['KAI', 'ABE', 'ADA', 'GUS', 'ZOE']
```

```
In [ ]: #Build a list and print the items in the list
dogs = []
dogs.append('willie')
dogs.append('hootz')
dogs.append('peso')
dogs.append('goblin')
for dog in dogs:
    print("Hello " + dog + "!")
print("I love these dogs!")
print("\nThese were my first two dogs:")
old_dogs = dogs[:2]
for old_dog in old_dogs:
    print(old_dog)
del dogs[0]
dogs.remove('peso')
print(dogs)
```

```
Hello willie!
Hello hootz!
Hello peso!
Hello goblin!
```

I love these dogs!

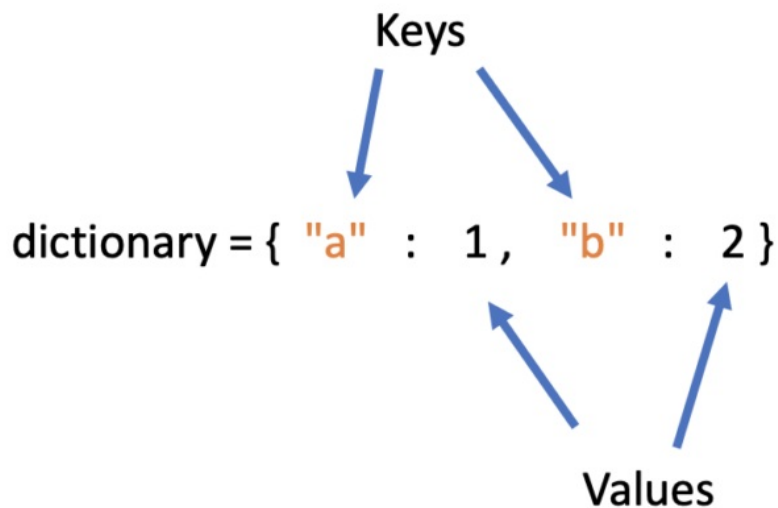
These were my first two dogs:

willie

hootz

['hootz', 'goblin']

Dictionaries



Use curly braces to define a dictionary. Use colons to connect keys and values, and use commas to separate individual key-value pairs.

```
In [ ]: # Making a dictionary
alien_0 = {'color': 'green', 'points': 5}
```

```
In [ ]: # Getting the value associated with a key
alien_0 = {'color': 'green', 'points': 5}
print(alien_0['color'])
print(alien_0['points'])
# Getting the value with get()
alien_0 = {'color': 'green'}
alien_color = alien_0.get('color')
alien_points = alien_0.get('points', 0)
print(alien_color)
print(alien_points)
```

```
green
5
green
0
```

```
In [ ]: # Adding a key-value pair
alien_0 = {'color': 'green', 'points': 5}
alien_0['x'] = 0
alien_0['y'] = 25
alien_0['speed'] = 1.5
print(alien_0)
# Adding to an empty dictionary
alien_0 = {}
alien_0['color'] = 'green'
alien_0['points'] = 5
print(alien_0)
```

```
{'color': 'green', 'points': 5, 'x': 0, 'y': 25, 'speed': 1.5}
{'color': 'green', 'points': 5}
```

```
In [ ]: #Modifying values in a dictionary
alien_0 = {'color': 'green', 'points': 5}
print(alien_0)
# Change the alien's color and point value.
alien_0['color'] = 'yellow'
alien_0['points'] = 10
print(alien_0)
```

```
{'color': 'green', 'points': 5}
{'color': 'yellow', 'points': 10}
```

```
In [ ]: # Deleting a key-value pair
alien_0 = {'color': 'green', 'points': 5}
print(alien_0)
del alien_0['points']
print(alien_0)
```

```
{'color': 'green', 'points': 5}
{'color': 'green'}
```

```
In [ ]: # Looping through all key-value pairs
# Store people's favorite languages.
fav_languages = {
    'jen': 'python',
    'sarah': 'c',
    'edward': 'ruby',
    'phil': 'python',
}
# Show each person's favorite language.
for name, language in fav_languages.items():
    print(name + ": " + language)
# Looping through all the keys
# Show everyone who's taken the survey.
for name in fav_languages.keys():
    print(name)
# Looping through all the values
# Show all the languages that have been chosen.
for language in fav_languages.values():
    print(language)
# Looping through all the keys in order
# Show each person's favorite language,
# in order by the person's name.
for name in sorted(fav_languages.keys()):
    print(name + ": " + language)
```

```
jen: python
sarah: c
edward: ruby
phil: python
jen
sarah
edward
phil
python
c
ruby
python
edward: python
jen: python
phil: python
sarah: python
```

```
In [ ]: # Finding a dictionary's length
num_responses = len(fav_languages)
num_responses
```

```
Out[ ]: 4
```

```
In [ ]: #Storing dictionaries in a list
# Start with an empty list.
```

```

users = []
# Make a new user, and add them to the list.
new_user = {
    'last': 'fermi',
    'first': 'enrico',
    'username': 'efermi',
}
users.append(new_user)
# Make another new user, and add them as well.
new_user = {
    'last': 'curie',
    'first': 'marie',
    'username': 'mcurie',
}
users.append(new_user)
# Show all information about each user.
for user_dict in users:
    for k, v in user_dict.items():
        print(k + ": " + v)
    print("\n")

```

```

last: fermi
first: enrico
username: efermi

```

```

last: curie
first: marie
username: mcurie

```

```

In [ ]: #You can also define a list of dictionaries directly,
# without using append():
# Define a list of users, where each user
# is represented by a dictionary.
users = [
    {
        'last': 'fermi',
        'first': 'enrico',
        'username': 'efermi',
    },
    {
        'last': 'curie',
        'first': 'marie',
        'username': 'mcurie',
    },
]
# Show all information about each user.
for user_dict in users:
    for k, v in user_dict.items():
        print(k + ": " + v)
    print("\n")

```

```

last: fermi
first: enrico
username: efermi

```

```

last: curie
first: marie
username: mcurie

```

```

In [ ]: # Storing lists in a dictionary
# Store multiple languages for each person.
fav_languages = {
    'jen': ['python', 'ruby'],
    'sarah': ['c'],
    'edward': ['ruby', 'go'],
    'phil': ['python', 'haskell'],
}
# Show all responses for each person.
for name, langs in fav_languages.items():
    print(name + ": ")
    for lang in langs:
        print("- " + lang)

```

```

jen:

```

```
- python
- ruby
sarah:
- c
edward:
- ruby
- go
phil:
- python
- haskell
```

```
In [ ]: # Storing dictionaries in a dictionary
users = {
    'aeinstein': {
        'first': 'albert',
        'last': 'einstein',
        'location': 'princeton',
    },
    'mcurie': {
        'first': 'marie',
        'last': 'curie',
        'location': 'paris',
    },
}
for username, user_dict in users.items():
    print("\nUsername: " + username)
    full_name = user_dict['first'] + " "
    full_name += user_dict['last']
    location = user_dict['location']
    print("\tFull name: " + full_name.title())
    print("\tLocation: " + location.title())
```

```
Username: aeinstein
    Full name: Albert Einstein
    Location: Princeton
```

```
Username: mcurie
    Full name: Marie Curie
    Location: Paris
```

```
In [ ]: # Preserving the order of keys and values
from collections import OrderedDict
# Store each person's languages, keeping
# track of who responded first.
fav_languages = OrderedDict()
fav_languages['jen'] = ['python', 'ruby']
fav_languages['sarah'] = ['c']
fav_languages['edward'] = ['ruby', 'go']
fav_languages['phil'] = ['python', 'haskell']
# Display the results, in the same order they
# were entered.
for name, langs in fav_languages.items():
    print(name + ":")
    for lang in langs:
        print("- " + lang)
```

```
jen:
- python
- ruby
sarah:
- c
edward:
- ruby
- go
phil:
- python
- haskell
```

```
In [ ]: # A million aliens
aliens = []
# Make a million green aliens, worth 5 points
# each. Have them all start in one row.
for alien_num in range(1000000):
    new_alien = {}
    new_alien['color'] = 'green'
    new_alien['points'] = 5
    new_alien['x'] = 20 * alien_num
```

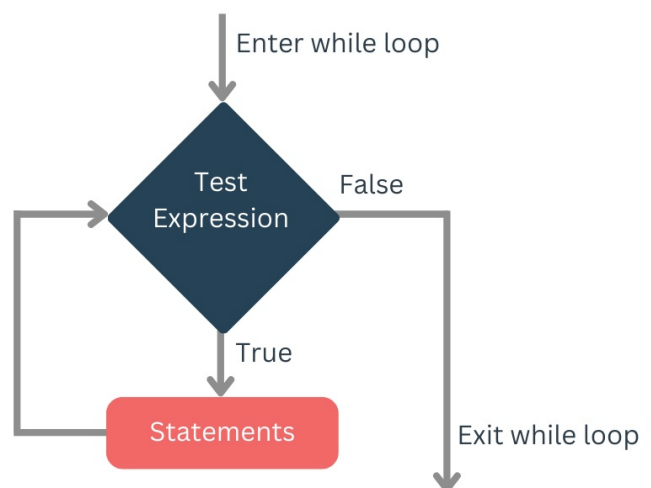
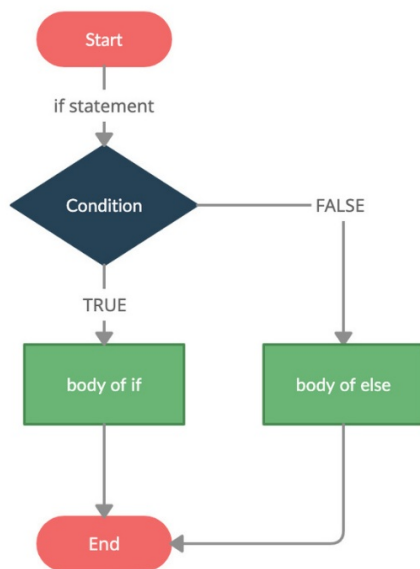
```

new_alien['y'] = 0
aliens.append(new_alien)
# Prove the list contains a million aliens.
num_aliens = len(aliens)
print("Number of aliens created:")
print(num_aliens)

```

Number of aliens created:
1000000

If Statements and While Loops



An if statement checks if an expression is true or false, and then runs the code inside the statement only if it is true. The code inside the loop is only run once. A while statement is a loop. Basically, it continues to execute the code in the while statement for however long the expression is true.

```

In [ ]: #conditional tests
#Checking for equality
#A single equal sign assigns a value to a variable. A double equal
#sign (==) checks whether two values are equal.
car = 'bmw'
print(car == 'bmw')

car = 'audi'
print(car == 'bmw')

# Ignoring case when making a comparison
car = 'Audi'
print(car.lower() == 'audi')

#Checking for inequality
topping = 'mushrooms'
print(topping != 'anchovies')

```

True
False
True
True

```

In [ ]: # Testing equality and inequality
age = 18
print(age == 18)
print(age != 18)

```

```
#Comparison operators
```

```
age = 19
print(age < 21)
print(age <= 21)
print(age > 21)
print(age >= 21)
```

```
True
False
True
True
False
False
```

```
In [ ]:
```

```
#Using and to check multiple conditions
```

```
age_0 = 22
age_1 = 18
print(age_0 >= 21 and age_1 >= 21)
age_1 = 23
print(age_0 >= 21 and age_1 >= 21)
```

```
#Using or to check multiple conditions
```

```
age_0 = 22
age_1 = 18
print(age_0 >= 21 or age_1 >= 21)
```

```
age_0 = 18
print(age_0 >= 21 or age_1 >= 21)
```

```
False
True
True
False
```

```
In [ ]:
```

```
#Simple boolean values
```

```
game_active = True
can_edit = False
```

```
In [ ]:
```

```
# if statements
```

```
#Simple if statement
```

```
age = 19
if age >= 18:
    print("You're old enough to vote!")
```

```
# If-else statements
```

```
age = 17
if age >= 18:
    print("You're old enough to vote!")
else:
    print("You can't vote yet.")
```

```
# The if-elif-else chain
```

```
age = 12
if age < 4:
    price = 0
elif age < 18:
    price = 5
else:
    price = 10
print("Your cost is $" + str(price) + ".")
```

```
You're old enough to vote!
You can't vote yet.
Your cost is $5.
```

```
In [ ]:
```

```
#Testing if a value is in a list
```

```
players = ['al', 'bea', 'cyn', 'dale']
print('al' in players)
print('eric' in players)
```

```
True
False
```

```
In [ ]:
```

```
# Testing if a value is not in a list
```

```
banned_users = ['ann', 'chad', 'dee']
```

```

user = 'erin'
if user not in banned_users:
    print("You can play!")
# Checking if a list is empty
players = []
if players:
    for player in players:
        print("Player: " + player.title())
else:
    print("We have no players yet!")

```

You can play!
We have no players yet!

```

In [ ]: #Simple input
name = input("What's your name? ")
print("Hello, " + name + ".")
#Accepting numerical input
age = input("How old are you? ")
age = int(age)
if age >= 18:
    print("\nYou can vote!")
else:
    print("\nYou can't vote yet.")

```

What's your name? vivek
Hello, vivek.
How old are you? 22

You can vote!

```

In [ ]: # While loops

```

```

In [ ]: # Counting to 5
current_number = 1
while current_number <= 5:
    print(current_number)
    current_number += 1

```

1
2
3
4
5

```

In [ ]: #Letting the user choose when to quit
prompt = "\nTell me something, and I'll "
prompt += "repeat it back to you."
prompt += "\nEnter 'quit' to end the program. "
message = ""
while message != 'quit':
    message = input(prompt)
    if message != 'quit':
        print(message)
#Using a flag
prompt = "\nTell me something, and I'll "
prompt += "repeat it back to you."
prompt += "\nEnter 'quit' to end the program. "
active = True
while active:
    message = input(prompt)
    if message == 'quit':
        active = False
    else:
        print(message)
# Using break to exit a loop
prompt = "\nWhat cities have you visited?"
prompt += "\nEnter 'quit' when you're done. "
while True:
    city = input(prompt)
    if city == 'quit':
        break
    else:
        print("I've been to " + city + "!")

```

Tell me something, and I'll repeat it back to you.

Enter 'quit' to end the program. quit

Tell me something, and I'll repeat it back to you.
Enter 'quit' to end the program. quit

What cities have you visited?
Enter 'quit' when you're done. quit

```
In [ ]: #Using continue in a loop
banned_users = ['eve', 'fred', 'gary', 'helen']
prompt = "\nAdd a player to your team."
prompt += "\nEnter 'quit' when you're done. "
players = []
while True:
    player = input(prompt)
    if player == 'quit':
        break
    elif player in banned_users:
        print(player + " is banned!")
        continue
    else:
        players.append(player)
        print("\nYour team:")
for player in players:
    print(player)
```

Add a player to your team.
Enter 'quit' when you're done. quit

```
In [ ]: #Removing all cats from a list of pets
pets = ['dog', 'cat', 'dog', 'fish', 'cat', 'rabbit', 'cat']
print(pets)
while 'cat' in pets:
    pets.remove('cat')

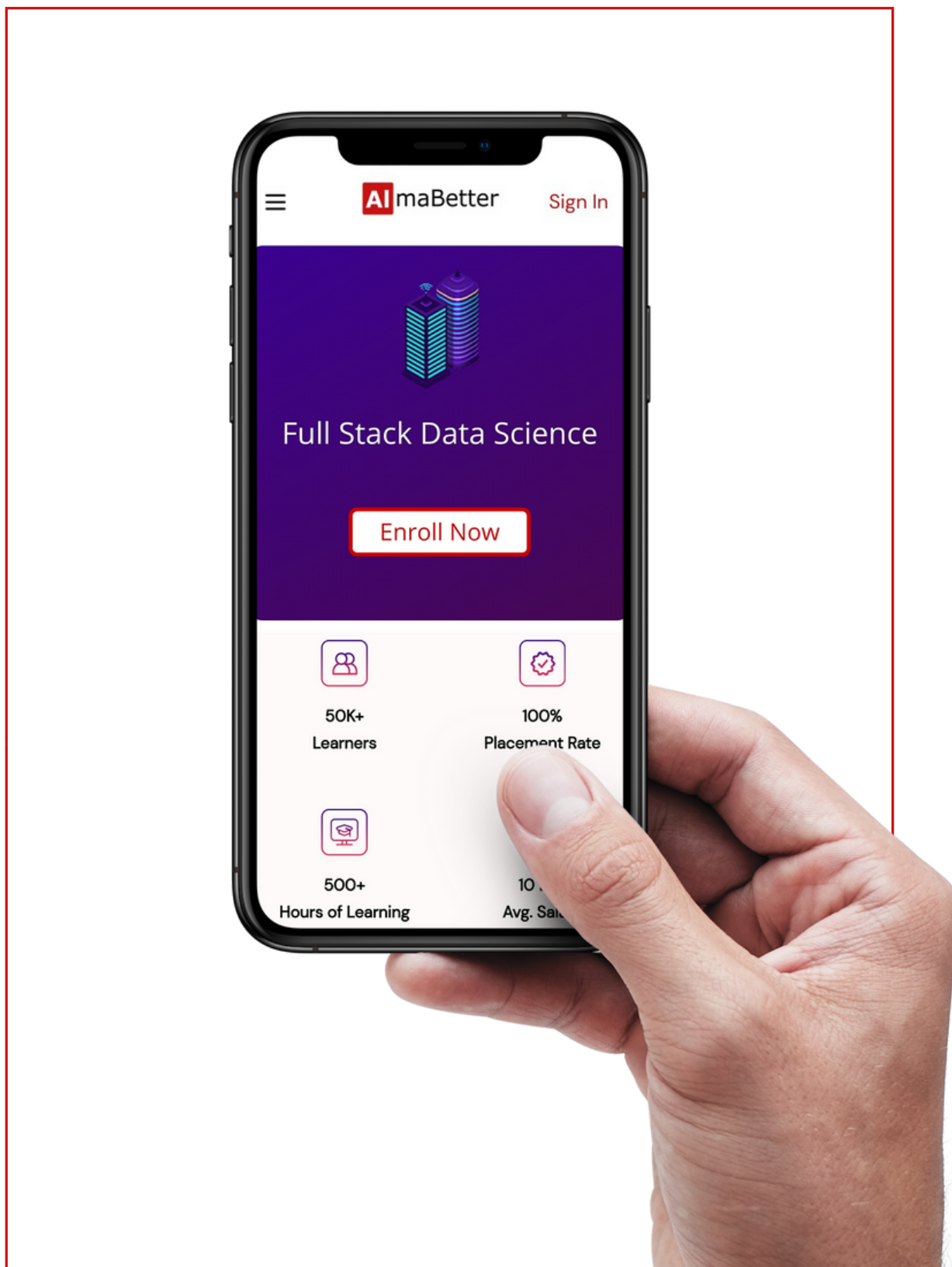
print(pets)
```

```
['dog', 'cat', 'dog', 'fish', 'cat', 'rabbit', 'cat']
['dog', 'dog', 'fish', 'rabbit']
```

```
In [ ]: # An infinite loop
#while True:
#     name = input("\nWho are you? ")
#     print("Nice to meet you, " + name + "!!")
```

```
In [ ]:
```

```
In [ ]:
```



If you're looking to get into **Data Science**, then **AlmaBetter** is the best place to start your journey.

Join our **Full Stack Data Science Program** and become a job-ready Data Science and Analytics professional in 30 weeks.

